



Escuela
Politécnica
Superior

Resolución de problemas de la Olimpiada Lingüística



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:

Alicja Wiktorja Hyska

Tutor/es:

Juan Antonio Pérez Ortiz

Mayo 2026



Universitat d'Alacant
Universidad de Alicante

Resolución de problemas de la Olimpiada Lingüística

Aplicación de modelos de inteligencia artificial

Autor

Alicja Wiktorja Hyska

Tutor/es

Juan Antonio Pérez Ortiz

Lenguajes y Sistemas Informáticos



Grado en Ingeniería Informática



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Mayo 2026

Preámbulo

Este trabajo surge del interés por explorar los límites del razonamiento lingüístico en los modelos de lenguaje actuales. El objetivo es diseñar y evaluar estrategias de prompting que permitan a estos modelos resolver puzzles de olimpiadas lingüísticas en idiomas nunca vistos durante su entrenamiento. El trabajo se enmarca en el área de procesamiento de lenguas de baja densidad de recursos y ha sido realizado bajo la supervisión de Juan Antonio Pérez Ortiz en el Grado de Ingeniería Informática de la Universidad de Alicante.

Agradecimientos

Quisiera agradecer, en primer lugar, a mi tutor, Juan Antonio Pérez Ortiz, por su orientación, sus consejos y su disponibilidad durante el desarrollo de este Trabajo de Fin de Grado. Su ayuda ha sido fundamental para dar forma al proyecto y para afrontar las distintas decisiones que han ido surgiendo a lo largo del proceso.

También quiero dar las gracias a mi familia, por su apoyo constante durante todos estos años. Su confianza, paciencia y ánimo han sido esenciales no solo durante la realización de este trabajo, sino a lo largo de toda mi etapa universitaria.

Este trabajo nace también de un interés personal por los idiomas y por la forma en que las lenguas permiten entender distintas culturas. Haber vivido en diferentes países me ha permitido acercarme a varios idiomas desde una perspectiva práctica y humana, y ha despertado en mí una curiosidad especial por la traducción y el razonamiento entre lenguas. Esa experiencia personal ha influido directamente en la elección de este tema, centrado en la resolución de problemas de olimpiadas lingüísticas mediante modelos de inteligencia artificial.

Por último, quiero agradecer a todas las personas que, de una forma u otra, me han acompañado durante este camino: compañeros, amigos y profesores que han contribuido a mi formación y me han ayudado a llegar hasta aquí.

El límite de mi lenguaje es el límite de mi mundo.

Ludwig Wittgenstein.

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Contexto y estado del problema	1
1.3. Estructura del documento	2
2. Marco Teórico y Estado de la Cuestión	3
2.1. Lenguas de baja densidad de recursos	3
2.2. Razonamiento metalingüístico	3
2.3. Benchmarks de olimpiadas lingüísticas para LLMs	4
2.3.1. PuzzLing Machines	4
2.3.2. modeLing	4
2.3.3. Linguini	4
2.3.4. LingOly	5
2.4. Estrategias de prompting para razonamiento lingüístico	5
2.4.1. Aprendizaje en contexto con ejemplos	5
2.4.2. Razonamiento paso a paso	5
2.4.3. Prompting analógico	6
2.4.4. Gramática en contexto	6
2.5. Análisis de las dificultades de los LLMs en puzzles lingüísticos	6
2.6. Síntesis y posicionamiento del trabajo	7
3. Objetivos	9
3.1. Objetivo principal	9
3.2. Objetivos específicos	9
3.3. Restricciones y condicionantes	10
4. Metodología y datos	11
4.1. Dataset: Linguini	11
4.1.1. Formato de los datos	11
4.2. Modelos evaluados	12
4.3. Estrategias de prompting	12
4.3.1. Instrucción metalingüística base	12
4.3.2. Estrategia 0: Baseline	13
4.3.3. Estrategia 1: Incremental por combinaciones	13
4.3.4. Estrategia 2: Step-by-step lingüístico	14
4.4. Sistema experimental	14
4.5. Métricas de evaluación	14
4.6. Análisis cualitativo con participantes humanos	15
4.6.1. Protocolo de resolución humana	15

4.6.2. Proceso de resolución observado	15
4.6.3. Comparación con el modelo	15
5. Desarrollo (Con ejemplos de código)	17
5.1. Inserción de código	17
5.2. Usos y personalización	20
5.3. Importar archivos fuente	21
6. Resultados (Con ejemplos de gráficos)	23
6.1. Diagramas	23
6.2. Gráficas	24
6.2.1. Línea	24
6.2.2. Barras	25
6.2.3. Polar	26
6.3. Importados de MATLAB	27
6.4. Ejemplo avanzado	28
7. Conclusiones (Con ejemplos de matemáticas)	31
7.1. Matemáticas	31
Bibliografía	35
A. Anexo I	39
B. Páginas horizontales	41
C. Importar PDF	45

Índice de figuras

6.1.	Diagrama realizado en latex con Tikz.	24
6.2.	Gráfica sencilla.	25
6.3.	OP/S003	25
6.4.	Gráfica barras.	26
6.5.	Directividad normalizada del altavoz (0 dBV en el eje).	27
6.6.	Ejemplo de gráfica obtenida con matlab2tikz.	28
6.7.	Amplitud de la aceleración en el modo número 8.	28
6.8.	Señal realizada con Tikz, sin imágenes.	29

Índice de tablas

4.1.	Puzzles utilizados en los experimentos	11
4.2.	Modelos evaluados	12

Índice de Códigos

5.1. ejemplo código C	17
5.2. ejemplo código C en color	18
5.3. ejemplo código PHP	18
5.4. ejemplo código PHP	18
5.5. ejemplo código Matlab en color	19
5.6. ejemplo código Matlab en blanco y negro	19
5.7. ejemplo código Python en color	20
5.8. ejemplo código Python en blanco y negro	20
5.9. Ejemplo de título abajo	21
5.10. Archivo C++ importado	22
5.11. Archivo Py importado	22
5.12. Archivo Matlab importado	22
6.1. Ejemplo de llamada a matlab2tikz	27

1. Introducción

1.1. Motivación

El lenguaje es una de las capacidades más complejas y distintivas de la especie humana. Se estima que en el mundo existen entre 6.000 y 7.000 lenguas vivas (Eberhard y cols., 2020), pero la gran mayoría de ellas son *lenguas de baja densidad de recursos* (*low-resource languages*): idiomas para los que apenas existen datos digitales, textos transcritos o herramientas de procesamiento automático. Esta escasez hace que los sistemas de inteligencia artificial convencionales, que dependen de grandes volúmenes de datos de entrenamiento, sean prácticamente inútiles para trabajar con ellas.

Sin embargo, los seres humanos somos capaces de aprender a leer, escribir y traducir en un idioma completamente desconocido a partir de un número reducido de ejemplos, utilizando nuestra *competencia metalingüística*: el conocimiento implícito sobre cómo funcionan las lenguas en general, sus patrones morfológicos, sus sistemas de concordancia, su tipología sintáctica. Esta capacidad, que demostramos cada vez que resolvemos un problema de la Olimpiada Lingüística, plantea una pregunta fundamental para la inteligencia artificial: ¿pueden los modelos de lenguaje de gran escala hacer lo mismo?

La Olimpiada Internacional de Lingüística (IOL) propone anualmente problemas del tipo *piedra Rosetta*: se presenta al participante un conjunto de frases en una lengua completamente desconocida junto con sus traducciones, y se le pide que deduzca las reglas del idioma y las aplique para traducir nuevas frases. Estos problemas son autocontenidos —toda la información necesaria está en el enunciado— y requieren razonamiento explícito, no memorización. Son, por tanto, un banco de pruebas ideal para medir si un sistema de inteligencia artificial posee razonamiento metalingüístico genuino.

Este trabajo parte de la hipótesis de que es posible diseñar estrategias de *prompting* que permitan a los modelos de lenguaje resolver estos problemas de forma progresiva, construyendo explícitamente un diccionario y un conjunto de reglas gramaticales a partir de los ejemplos proporcionados, sin recibir ninguna pista sobre la lengua en cuestión. Si esta hipótesis se confirma, las implicaciones son significativas: abriría la puerta a utilizar modelos de lenguaje para procesar, documentar y analizar miles de lenguas del mundo para las que no existe ni existirá suficiente cantidad de datos de entrenamiento.

1.2. Contexto y estado del problema

En los últimos años, los modelos de lenguaje de gran escala (*Large Language Models*, LLMs) han demostrado capacidades notables en tareas de razonamiento, traducción automática y comprensión del lenguaje natural (Brown y cols., 2020). Sin embargo, su rendimiento en lenguas de baja densidad de recursos sigue siendo considerablemente inferior al que alcanzan en lenguas de alta densidad como el inglés, el español o el chino (Joshi y cols., 2020).

El benchmark LINGUINI (Sánchez y cols., 2024), publicado por investigadores de Meta AI, la University College London y la Universidad del País Vasco, constituye actualmente la evaluación más completa y rigurosa de la capacidad de razonamiento lingüístico de los LLMs sobre lenguas no vistas. Sus resultados son reveladores: ningún modelo analizado supera el 25% de exactitud, con el mejor modelo propietario (Claude 3 Opus) alcanzando un 24,05% y el mejor modelo de código abierto (Llama 3 70B) un 8,84%. Estos valores contrastan marcadamente con el rendimiento humano, que en competición real supera con frecuencia el 60%.

Trabajos anteriores han explorado estrategias para mejorar este rendimiento. Zhu y cols. (2025) proponen una descomposición paso a paso de los puzzles lingüísticos, presentando al modelo las reglas de forma incremental y obteniendo mejoras significativas respecto al prompting directo. Tanzer y cols. (2024) demuestran que los LLMs pueden aprender a traducir una lengua no vista a partir de un único libro de gramática utilizado como contexto. Estos resultados apuntan a que la clave no está en el tamaño del modelo sino en la *estrategia de presentación de la información*.

1.3. Estructura del documento

El presente documento se organiza en los siguientes capítulos:

El **Capítulo ??** presenta el estado de la cuestión, revisando los fundamentos teóricos del razonamiento metalingüístico, los principales benchmarks de olimpiadas lingüísticas para LLMs y las estrategias de prompting más relevantes de la literatura.

El **Capítulo 3** establece el objetivo principal del trabajo y los objetivos específicos que se derivan de él, así como las restricciones y condicionantes del proyecto.

El **Capítulo 4** describe la metodología del trabajo: los puzzles utilizados, los modelos evaluados, las tres estrategias de prompting diseñadas, el sistema experimental implementado y las métricas de evaluación empleadas.

El **Capítulo ??** presenta y analiza los resultados experimentales obtenidos, incluyendo la comparación con los valores de referencia del benchmark LINGUINI y el análisis cualitativo comparando el razonamiento del modelo con el humano.

El **Capítulo ??** recoge las conclusiones del trabajo, sus limitaciones y las líneas de trabajo futuro que se abren a partir de los resultados obtenidos.

Finalmente, los anexos incluyen el código fuente del sistema experimental y ejemplos representativos de las interacciones entre los modelos y los puzzles lingüísticos.

2. Marco Teórico y Estado de la Cuestión

Este capítulo revisa el estado del arte en los campos que confluyen en este trabajo: el procesamiento de lenguas de baja densidad de recursos, el razonamiento metalingüístico en modelos de lenguaje de gran escala, los benchmarks basados en olimpiadas lingüísticas y las estrategias de prompting más relevantes para la tarea abordada.

2.1. Lenguas de baja densidad de recursos

Se estima que existen entre 6.000 y 7.000 lenguas vivas en el mundo (Eberhard y cols., 2020), pero la inmensa mayoría de ellas carecen de los recursos digitales necesarios para desarrollar sistemas de procesamiento automático del lenguaje natural (PLN). Joshi y cols. (2020) cuantifican esta desigualdad mostrando que más del 88% de las lenguas del mundo apenas tienen representación en los conjuntos de datos utilizados para entrenar los modelos actuales. Estas lenguas se denominan *lenguas de baja densidad de recursos* (*low-resource languages*, LRL) y su estudio plantea uno de los retos más relevantes del PLN contemporáneo.

La escasez de datos tiene consecuencias directas en el rendimiento de los sistemas de traducción automática, reconocimiento de voz y análisis morfosintáctico para estas lenguas. Los modelos de lenguaje actuales, entrenados mayoritariamente sobre texto en inglés y otras lenguas de alta densidad, presentan un rendimiento muy inferior cuando se enfrentan a lenguas con poca presencia en sus datos de entrenamiento (Joshi y cols., 2020). Este desequilibrio no es únicamente un problema técnico: tiene implicaciones directas para la preservación del patrimonio lingüístico y cultural de comunidades hablantes de lenguas minorizadas.

Una aproximación prometedora para abordar este problema es el aprendizaje en contexto (*in-context learning*, ICL): la capacidad de los modelos de lenguaje de gran escala para realizar tareas nuevas a partir de unos pocos ejemplos proporcionados directamente en el prompt, sin necesidad de reentrenamiento. Esta capacidad abre la posibilidad de trabajar con lenguas para las que no existen datos de entrenamiento suficientes, siempre que el modelo sea capaz de razonar sobre los patrones lingüísticos presentes en los ejemplos.

2.2. Razonamiento metalingüístico

El concepto de *competencia metalingüística* hace referencia a la capacidad de razonar sobre el lenguaje como objeto de estudio, más allá del uso comunicativo cotidiano. Un hablante con alta competencia metalingüística es capaz de identificar patrones morfológicos en una lengua desconocida, deducir reglas gramaticales a partir de ejemplos y aplicarlas a nuevas frases, sin necesidad de haber aprendido previamente ese idioma.

Esta capacidad es precisamente la que explotan los problemas de la Olimpiada Internacional de Lingüística (IOL), celebrada anualmente desde 2003. Los participantes —estudiantes de secundaria— deben resolver problemas del tipo *piedra Rosetta* (Bozhanov y Derzhanski,

2013): se les presenta un conjunto de frases en una lengua completamente desconocida junto con sus traducciones al inglés, y deben deducir las reglas del idioma para traducir nuevas frases. Los problemas son autocontenidos —toda la información necesaria está en el enunciado— y están diseñados para lenguas de baja densidad de recursos, lo que minimiza la posibilidad de que los participantes tengan conocimiento previo del idioma (Derzhanski y Payne, 2010).

La pregunta central que motiva este TFG y que ha generado una creciente línea de investigación en los últimos años es la siguiente: ¿pueden los modelos de lenguaje de gran escala desarrollar o simular esta competencia metalingüística? La respuesta que ofrece la literatura es que, aunque los modelos actuales muestran cierta capacidad de razonamiento lingüístico inductivo, su rendimiento dista considerablemente del humano en tareas que requieren deducción sistemática de reglas en lenguas no vistas.

2.3. Benchmarks de olimpiadas lingüísticas para LLMs

La evaluación de la capacidad de razonamiento lingüístico en LLMs ha dado lugar a varios benchmarks específicos en los últimos años, todos ellos basados en problemas de olimpiadas lingüísticas. A continuación se describen los más relevantes para este trabajo.

2.3.1. PuzzLing Machines

El trabajo de Şahin y cols. (2020) constituye el antecedente directo de toda esta línea de investigación. Los autores presentan un conjunto de siete patrones de problemas de tipo piedra Rosetta extraídos de la IOL, y evalúan la capacidad de modelos de traducción automática neural para resolverlos. Sus resultados muestran que los sistemas convencionales de traducción automática fracasan sistemáticamente en esta tarea, ya que no están diseñados para aprender una lengua nueva a partir de unos pocos ejemplos. Este trabajo establece el marco conceptual que seguirán los trabajos posteriores.

2.3.2. modeLing

Chi y cols. (2024) presentan modeLing, un dataset construido manualmente con problemas similares a los de la IOL, diseñado específicamente para evitar la contaminación de datos de entrenamiento. A diferencia de los benchmarks basados directamente en el corpus de la IOL, los problemas de modeLing son originales, lo que garantiza que los modelos evaluados no puedan haberlos memorizado durante el preentrenamiento. El dataset se centra en problemas de transducción de secuencias y ha sido utilizado como base de comparación en trabajos posteriores.

2.3.3. Linguini

Sánchez y cols. (2024) presentan Linguini, el benchmark que constituye la referencia central de este trabajo. Linguini está compuesto por 894 preguntas agrupadas en 160 problemas extraídos del corpus de la IOL, cubriendo 75 lenguas de baja densidad de recursos. Los problemas se clasifican en tres categorías: transducción de secuencias (traducción), relleno de huecos morfológicos y transliteración numérica.

Los resultados de la evaluación son reveladores: ninguno de los modelos analizados supera el 25% de exactitud. El mejor modelo propietario, Claude 3 Opus, alcanza un 24,05% de exactitud y un chrF++ de 63,96 en configuración zero-shot. El mejor modelo de código abierto, Llama 3 70B, obtiene un 8,84% de exactitud y un chrF++ de 37,25. Estos valores contrastan con el rendimiento humano en competición, que supera habitualmente el 60%.

Un hallazgo especialmente relevante del paper es el experimento de ablación sin contexto: al eliminar las frases de entrenamiento del prompt, el rendimiento de todos los modelos cae drásticamente, lo que confirma que los modelos resuelven los puzzles por razonamiento inductivo sobre el contexto proporcionado, no por memorización del idioma durante el preentrenamiento.

2.3.4. LingOly

Bean y cols. (2024) presentan LingOly, el benchmark más exhaustivo del campo hasta la fecha, publicado en NeurIPS 2024. LingOly cubre más de 90 lenguas —incluyendo algunas extintas— y contiene 1.133 problemas organizados en seis categorías lingüísticas (fonología, morfología, sintaxis, semántica, sistemas numéricos y problemas compuestos) y cinco niveles de dificultad humana. Esta riqueza taxonómica permite análisis más granulares que los benchmarks anteriores.

Los resultados en LingOly confirman y amplían los de Linguini: el rendimiento de los modelos es consistentemente bajo, especialmente en los problemas de mayor dificultad, donde incluso el mejor modelo alcanza solo el 38,7% de exactitud con una mejora de 24,7 puntos sobre el baseline sin contexto. Los autores observan además que los modelos cerrados superan consistentemente a los abiertos, y que existe una correlación positiva entre la densidad de recursos de una lengua y el rendimiento del modelo sobre ella.

2.4. Estrategias de prompting para razonamiento lingüístico

Ante los pobres resultados obtenidos con prompting directo, varios trabajos han explorado estrategias más sofisticadas para mejorar la capacidad de los LLMs en esta tarea.

2.4.1. Aprendizaje en contexto con ejemplos

La estrategia más simple consiste en proporcionar al modelo ejemplos adicionales del mismo formato de tarea antes de presentar el problema a resolver (*few-shot learning*). Sánchez y cols. (2024) evalúan configuraciones de 0 a 5 ejemplos en contexto y observan un efecto mixto: algunos modelos mejoran con más ejemplos, mientras que otros empeoran, posiblemente porque los ejemplos adicionales introducen ruido de lenguas no relacionadas con el puzzle en cuestión.

2.4.2. Razonamiento paso a paso

Zhu y cols. (2025) proponen una estrategia de descomposición paso a paso inspirada en el chain-of-thought prompting (Wei y cols., 2022). En lugar de presentar todos los ejemplos de entrenamiento de golpe, organizan la información en etapas lingüísticas sucesivas: primero se presenta el léxico y el orden de palabras básico, después los patrones fonológicos, luego la

morfosintaxis y finalmente la sintaxis compleja. En cada etapa, el modelo actualiza su comprensión de la lengua antes de pasar a la siguiente. Este enfoque obtiene mejoras significativas respecto al prompting directo en lenguas con morfología verbal compleja.

2.4.3. Prompting analógico

Ramji y Ramji (2024) proponen una estrategia diferente basada en el prompting analógico: en lugar de depender únicamente de los ejemplos del puzzle, el modelo genera automáticamente ejemplos auxiliares en lenguas tipológicamente relacionadas que conoce de su preentrenamiento, y los utiliza como contexto adicional para guiar el razonamiento inductivo. Esta aproximación explota el conocimiento implícito del modelo sobre familias de lenguas para compensar la escasez de ejemplos en la lengua objetivo.

2.4.4. Gramática en contexto

Tanzer y cols. (2024) demuestran que los LLMs pueden adquirir cierta competencia traductora en una lengua no vista si se les proporciona directamente un libro de gramática como contexto. Este enfoque, denominado *one-book prompting*, no requiere pares de traducción y es especialmente interesante para lenguas que disponen de gramáticas descriptivas pero no de corpus paralelos. Los resultados muestran mejoras modestas pero consistentes sobre el baseline sin contexto, y apuntan a que la combinación de gramática y ejemplos es más efectiva que cualquiera de los dos por separado.

2.5. Análisis de las dificultades de los LLMs en puzzles lingüísticos

Más allá de cuantificar el rendimiento de los modelos, algunos trabajos recientes han intentado explicar *por qué* los LLMs fallan en estos puzzles.

Choudhary y cols. (2025) realizan el análisis más sistemático hasta la fecha, anotando 629 problemas en 41 lenguas de baja densidad con 50 características lingüísticas y correlacionando estas características con el rendimiento de los modelos. Sus conclusiones principales son dos. En primer lugar, la complejidad morfológica es el factor predictivo más importante del fracaso: cuanto más compleja es la morfología de la lengua (más morfemas por palabra, más procesos de afijación y fusión), peor rinden los modelos. En segundo lugar, los modelos rinden significativamente mejor en puzzles cuyas características lingüísticas también están presentes en inglés, lo que sugiere que el razonamiento metalingüístico de los LLMs está sesgado hacia los patrones del inglés y las lenguas de alta densidad de recursos.

Este hallazgo tiene implicaciones directas para la interpretación de los resultados de este trabajo: la variación en el rendimiento entre los distintos puzzles evaluados puede explicarse en parte por las diferencias tipológicas entre las lenguas seleccionadas.

El trabajo más reciente del campo, Majmudar y Filatova (2025), extiende la evaluación a modelos de razonamiento como o1 de OpenAI y encuentra que estos modelos superan a los humanos en la mayoría de categorías de puzzles, con la excepción notable de los sistemas de escritura no familiares y las lenguas más alejadas tipológicamente del inglés. Este resultado abre una nueva perspectiva sobre el papel del razonamiento explícito en la resolución de puzzles lingüísticos, y sugiere que los modelos con capacidad de razonamiento extendido representan el siguiente paso en esta línea de investigación.

2.6. Síntesis y posicionamiento del trabajo

La revisión de la literatura permite identificar con claridad el hueco que ocupa este TFG. Los trabajos existentes han establecido que los LLMs son capaces de razonamiento lingüístico inductivo pero con un rendimiento muy inferior al humano; han identificado la complejidad morfológica como el principal factor de dificultad; y han explorado estrategias de prompting que mejoran el rendimiento con respecto al baseline directo.

Sin embargo, ningún trabajo previo ha evaluado sistemáticamente una estrategia de presentación *incremental por combinaciones crecientes*: proporcionar al modelo subconjuntos de ejemplos de tamaño creciente —de dos en dos, de tres en tres, etc.— para que construya y refine iterativamente su propio diccionario y sus propias reglas gramaticales, con capacidad explícita de corregir y eliminar hipótesis anteriores conforme recibe nueva evidencia. Esta estrategia se inspira en el proceso cognitivo humano de aprendizaje de lenguas desconocidas —tal y como se documenta en los protocolos de resolución de puzzles de olimpiada— y constituye la aportación metodológica principal de este trabajo.

Adicionalmente, este trabajo compara modelos de dos familias distintas (Llama de Meta y Gemma de Google) en dos tamaños diferentes, y utiliza chrF++ como métrica principal de evaluación, en línea con las recomendaciones más recientes de la literatura para tareas de traducción con lenguas de morfología compleja (Popović, 2017).

3. Objetivos

Este capítulo presenta el objetivo principal del trabajo y los objetivos específicos que se derivan de él, así como las restricciones y condicionantes que han guiado el desarrollo del proyecto.

3.1. Objetivo principal

El objetivo principal de este Trabajo de Fin de Grado es diseñar, implementar y evaluar estrategias de *prompting* para que modelos de lenguaje de gran escala puedan resolver puzzles de olimpiadas lingüísticas en idiomas que nunca han formado parte de su entrenamiento, apoyándose únicamente en razonamiento metalingüístico y sin recibir ninguna pista sobre la lengua en cuestión.

3.2. Objetivos específicos

De este objetivo principal se derivan los siguientes objetivos específicos:

1. **Revisar el estado del arte** en razonamiento lingüístico con modelos de lenguaje de gran escala, con especial atención a los benchmarks derivados de la Olimpiada Internacional de Lingüística (LINGUINI, LingOly, modeLing) y a las estrategias de prompting más relevantes de la literatura reciente.
2. **Diseñar e implementar tres estrategias de prompting** diferenciadas y comparables entre sí:
 - Una estrategia *baseline* que sirva como punto de referencia directo con la literatura existente.
 - Una estrategia de presentación incremental por combinaciones crecientes de ejemplos, con actualización explícita y revisable del diccionario y las reglas gramaticales.
 - Una estrategia paso a paso ordenada por nivel lingüístico, inspirada en el trabajo de Zhu y cols. (2025).
3. **Construir un sistema experimental reproducible** que automatice la ejecución de los experimentos, el registro completo de los prompts y respuestas, y el cálculo de métricas, permitiendo auditar cualquier resultado.
4. **Evaluar las estrategias** sobre puzzles reales del benchmark LINGUINI en lenguas de cuatro familias lingüísticas distintas, utilizando modelos de dos familias arquitectónicas diferentes y tres tamaños.

5. **Comparar los resultados obtenidos** con los valores de referencia publicados en Sánchez y cols. (2024) mediante las métricas BLEU y chrF++, identificando en qué condiciones cada estrategia supera o iguala el estado del arte.
6. **Realizar un análisis cualitativo** comparando el proceso de resolución humana de los puzzles con el razonamiento explícito del modelo, identificando los puntos de divergencia y los patrones de error más frecuentes.

3.3. Restricciones y condicionantes

El desarrollo del trabajo está sujeto a las siguientes restricciones:

Acceso a los modelos: se utilizan exclusivamente modelos accesibles mediante APIs gratuitas (Groq y Google AI Studio), lo que limita los modelos disponibles pero garantiza la reproducibilidad del trabajo sin coste computacional.

Límites de tasa: las APIs gratuitas imponen límites de peticiones por minuto y por día, lo que condiciona el número de experimentos que pueden ejecutarse en una sesión y obliga a introducir retardos entre llamadas.

Datos: los puzzles utilizados provienen exclusivamente del benchmark LINGUINI (Sánchez y cols., 2024), cuya descarga requiere aceptar los términos de uso del corpus de la IOL (licencia CC-BY-SA 4.0). No se utilizan puzzles de elaboración propia para mantener la comparabilidad con la literatura.

Ausencia de reentrenamiento: no se realiza ningún tipo de *fine-tuning* ni adaptación de los modelos. Todas las estrategias operan en modo *zero-shot* o *few-shot* mediante prompting, sin modificar los pesos del modelo.

4. Metodología y datos

Este capítulo describe el diseño experimental del trabajo: el dataset utilizado, los modelos evaluados, las tres estrategias de prompting implementadas, el sistema de evaluación automática y el protocolo de análisis cualitativo con participantes humanos.

4.1. Dataset: Linguini

Los experimentos de este trabajo utilizan puzzles extraídos del benchmark LINGUINI (Sánchez y cols., 2024), que a su vez provienen del corpus de la Olimpiada Internacional de Lingüística (IOL). Se seleccionaron cuatro puzzles de tipo *sequence transduction* (traducción), que es la categoría más relevante para evaluar razonamiento metalingüístico inductivo y la única que permite comparación directa con el trabajo de Zhu y cols. (2025).

Los criterios de selección de los cuatro puzzles fueron los siguientes: que pertenecieran a familias lingüísticas distintas entre sí, para maximizar la diversidad tipológica; que tuvieran al menos 10 pares de entrenamiento, para que la estrategia incremental tuviera margen suficiente para construir conocimiento progresivo; y que la dirección de traducción fuera de la lengua desconocida al inglés, que es el formato más frecuente en la IOL.

La Tabla 4.1 resume las características de los cuatro puzzles seleccionados, junto con el puzzle piloto de Ayutla Mixe utilizado durante el desarrollo del sistema experimental.

Tabla 4.1: Puzzles utilizados en los experimentos

ID	Lengua	Familia	Región	Train	Test
piloto	Ayutla Mixe	Mixe-Zoquean	México	8	4
012018020100	Hakhun	Sino-Tibetana	India	10	6
012023020100	Apurinã	Maipurean	Brasil	16	3
012008040100	Copainalá Zoque	Mixe-Zoquean	México	20	6
012013010200	Yidiny	Pama-Nyungan	Australia	23	4

Todos los puzzles están formados por pares de la forma (**frase en lengua desconocida, traducción al inglés**). Los pares de entrenamiento se proporcionan al modelo como contexto, y los pares de test se utilizan únicamente para evaluar las traducciones generadas, sin que el modelo tenga acceso a sus respuestas correctas durante la inferencia.

4.1.1. Formato de los datos

Los puzzles del benchmark LINGUINI se distribuyen en formato JSONL. Para este trabajo se implementó un conversor que transforma cada problema al formato JSON propio del sistema experimental, con los campos `train_pairs`, `test_pairs`, `language`, `language_family`

y **source**. Este formato estandarizado permite añadir nuevos puzzles al sistema de forma sencilla y garantiza la reproducibilidad de los experimentos.

4.2. Modelos evaluados

Se evaluaron modelos de dos familias distintas y tres tamaños diferentes, todos ellos accesibles mediante APIs gratuitas, lo que garantiza la reproducibilidad del trabajo sin coste computacional. La Tabla 4.2 resume los modelos utilizados.

Tabla 4.2: Modelos evaluados

Denominación	Modelo	Familia	Parámetros	Tier	API
Llama-8B	Llama 3.1 8B Instant	Meta / Llama	8B	Débil	Groq
Llama-70B	Llama 3.3 70B Versatile	Meta / Llama	70B	Medio	Groq
Gemma-4	Gemma 4	Google / Gemma	–	Fuerte	Google AI Studio

La elección de dos modelos de la misma familia (Llama 3.x) en tamaños distintos permite aislar el efecto del tamaño del modelo manteniendo constante la arquitectura y el proceso de preentrenamiento. La inclusión de Gemma 4, de una familia completamente distinta, añade diversidad arquitectónica a la comparación y permite evaluar si los patrones observados son específicos de los modelos Llama o generalizables a otras familias.

Todos los modelos se utilizan en modo *zero-shot*: no se les proporcionan ejemplos del formato de respuesta esperado, solo las instrucciones del prompt y los ejemplos del puzzle. La temperatura se fija a 0 en todos los casos para garantizar la reproducibilidad de los resultados.

4.3. Estrategias de prompting

Se diseñaron e implementaron tres estrategias de prompting, descritas a continuación. Las tres comparten una instrucción metalingüística base que establece el marco de razonamiento del modelo.

4.3.1. Instrucción metalingüística base

Las tres estrategias parten de una instrucción común que instruye al modelo a actuar como analista lingüístico que nunca ha visto la lengua objetivo, a razonar exclusivamente a partir de los ejemplos proporcionados y a tratar todas sus hipótesis como provisionales y revisables. Específicamente, se le indica que no utilice conocimiento previo sobre la lengua o su familia lingüística, que construya explícitamente un diccionario y un conjunto de reglas gramaticales, y que muestre su razonamiento antes de alcanzar cualquier conclusión. Esta instrucción se inspira en el enfoque metalingüístico descrito por Zhu y cols. (2025) y en los principios del *chain-of-thought prompting* (Wei y cols., 2022).

4.3.2. Estrategia 0: Baseline

La estrategia baseline proporciona al modelo todos los pares de entrenamiento del puzzle en un único prompt y le pide que genere directamente el diccionario, las reglas gramaticales y las traducciones de las frases de test. Esta estrategia replica el prompting directo utilizado como referencia en Sánchez y cols. (2024) y sirve como punto de comparación para las demás estrategias.

El prompt tiene la siguiente estructura:

1. Instrucción metalingüística base
2. Todos los pares de entrenamiento
3. Solicitud de diccionario, reglas y traducciones

4.3.3. Estrategia 1: Incremental por combinaciones

La estrategia incremental constituye la aportación metodológica principal de este trabajo y está inspirada en el proceso cognitivo humano de resolución de puzzles lingüísticos. En lugar de proporcionar todos los ejemplos de golpe, se presentan subconjuntos de tamaño creciente: primero todas las combinaciones de 2 frases ($\binom{n}{2}$), luego de 3 ($\binom{n}{3}$), y así sucesivamente hasta presentar todas las n frases de entrenamiento.

En cada paso, el modelo recibe el subconjunto de frases correspondiente junto con su hipótesis acumulada del paso anterior (diccionario y reglas), y debe realizar cuatro operaciones en orden:

1. **Análisis** (*STEP A*): razonar explícitamente sobre los nuevos ejemplos antes de actualizar ninguna hipótesis, identificando morfemas, detectando contradicciones con el conocimiento previo y explicando el razonamiento.
2. **Diccionario revisado** (*STEP B*): actualizar el diccionario añadiendo entradas nuevas, corrigiendo entradas imprecisas y eliminando entradas refutadas (marcadas como [DELETED: razón]).
3. **Reglas revisadas** (*STEP C*): actualizar el conjunto de reglas gramaticales con la misma lógica de añadir, corregir y eliminar.
4. **Traducciones** (*STEP D*): traducir las frases de test utilizando únicamente el diccionario y las reglas revisadas.

La capacidad explícita de eliminar hipótesis erróneas (*DELETE*) es un elemento diferencial respecto a las estrategias de la literatura, donde el conocimiento acumulado solo crece. Esta decisión de diseño se fundamenta en la observación de que los modelos tienden a mantener hipótesis incorrectas incluso cuando reciben evidencia contradictoria, si el prompt no les obliga explícitamente a revisarlas (Zhu y cols., 2025).

4.3.4. Estrategia 2: Step-by-step lingüístico

La estrategia step-by-step está inspirada en el trabajo de Zhu y cols. (2025) y organiza la presentación de los ejemplos según una jerarquía lingüística de cuatro etapas:

1. **Léxico** (35% de los ejemplos): frases simples que permiten identificar vocabulario básico y el orden de palabras fundamental.
2. **Fonología** (15% de los ejemplos): frases que exhiben variaciones fonológicas (alternancias vocálicas, alófonos, procesos de asimilación).
3. **Morfosintaxis** (30% de los ejemplos): frases con concordancia de persona, número, tiempo y aspecto.
4. **Sintaxis** (20% de los ejemplos): frases con estructuras más complejas (negación, preguntas indirectas, cláusulas subordinadas).

Esta partición es heurística cuando los puzzles no disponen de anotación lingüística explícita, que es el caso de los puzzles de LINGUINI. Al igual que en la estrategia incremental, el modelo acumula diccionario y reglas entre etapas y puede corregir hipótesis previas.

4.4. Sistema experimental

Se implementó un sistema experimental completo en Python que automatiza la ejecución de los experimentos, el cálculo de métricas y el registro de resultados. El sistema consta de tres módulos principales.

El módulo `experiment_runner.py` es el motor central. Dado un puzzle y un modelo, ejecuta las tres estrategias de prompting, calcula las métricas en cada paso y guarda un log completo en formato JSON. Cada log incluye el prompt exacto enviado al modelo, la respuesta completa recibida, las traducciones extraídas, las métricas calculadas y el diccionario y reglas acumulados en cada paso. Este registro exhaustivo permite reproducir y auditar cualquier experimento.

El módulo `generate_report.py` lee los logs JSON y genera tablas comparativas en formato Markdown. El módulo `setup_and_test.py` verifica la configuración de las APIs y realiza llamadas de prueba antes de lanzar los experimentos completos.

Las llamadas a los modelos se realizan a través de las APIs de Groq y Google AI Studio, ambas compatibles con el protocolo OpenAI, lo que simplifica la integración. Se utiliza una temperatura de 0 en todas las llamadas para garantizar la reproducibilidad, y un retardo de 10 segundos entre llamadas para respetar los límites de tasa de las APIs gratuitas.

4.5. Métricas de evaluación

Se utilizan dos métricas automáticas de evaluación, calculadas con la biblioteca `sacrebleu` (Post, 2018):

BLEU (Papineni y cols., 2002) es la métrica estándar en traducción automática. Calcula la precisión de n-gramas compartidos entre la hipótesis del modelo y la referencia correcta, con una penalización por brevedad. Su principal limitación en este contexto es que requiere

coincidencia exacta de palabras, lo que la hace excesivamente estricta para lenguas con morfología compleja: una traducción casi correcta que difiera en un sufijo verbal recibirá un score de 0. Se incluye por compatibilidad con la literatura existente.

chrF++ (Popović, 2017) es la métrica principal de este trabajo. A diferencia de BLEU, opera a nivel de n-gramas de caracteres y de palabras (parámetro `word_order=2`), lo que la hace más tolerante con variaciones morfológicas. Es más adecuada para lenguas aglutinantes y de morfología compleja, y es la métrica secundaria utilizada por el propio benchmark LINGUINI (Sánchez y cols., 2024). Un modelo que casi acierta una forma verbal pero falla en un sufijo de concordancia recibirá un chrF++ parcial, mientras que BLEU le daría 0.

4.6. Análisis cualitativo con participantes humanos

Además de la evaluación automática, se realizó un análisis cualitativo comparando el proceso de resolución humano con el del modelo. El objetivo es identificar en qué aspectos del razonamiento metalingüístico los modelos divergen de los humanos, más allá de lo que capturan las métricas automáticas.

4.6.1. Protocolo de resolución humana

Se seleccionó el puzzle de Hakhun (Sino-Tibetana, India; 10 frases de entrenamiento, 6 de test) para el análisis cualitativo. El protocolo consistió en resolver el puzzle anotando explícitamente cada paso del razonamiento: qué patrones se identificaron primero, qué hipótesis se formularon, cuándo y por qué se corrigieron, y cómo se aplicaron a las frases de test. Se probaron además dos condiciones de presentación: ver las frases de dos en dos (condición incremental) y ver todas las frases de golpe (condición baseline), para comparar cómo cambia el proceso cognitivo.

4.6.2. Proceso de resolución observado

En la condición baseline (todas las frases de golpe), el proceso humano siguió el siguiente orden: primero se identificaron los elementos invariantes (**ne** como marcador de pregunta, presente en todas las frases), luego los pronombres de sujeto (**ṇa**=I, **n**=you(sg), **nirum**=we, **tarum**=they), después los verbos (**lapk i**=see, **c am**=know, **lan**=beat, **ka**=go, **ip**=sleep), y finalmente el sistema aspectual: las terminaciones con **o t** - marcan pasado, las demás marcan presente. El marcador **kəmə** se identificó como marcador de transitividad sin traducción directa. Con este protocolo se obtuvieron las seis traducciones de test correctas.

4.6.3. Comparación con el modelo

La comparación con la respuesta del modelo en la estrategia baseline revela un patrón consistente: el modelo identifica correctamente el léxico básico (verbos y pronombres de sujeto) pero falla sistemáticamente en el sistema aspectual (pasado vs. presente). El modelo menciona en su razonamiento la posibilidad de que **o t** marque pasado, pero no lo aplica de forma consistente a todas las formas verbales. Este resultado está en línea con el hallazgo de Choudhary y cols. (2025) de que los LLMs rinden peor en puzzles con mayor complejidad

morfológica, ya que la concordancia de tiempo, aspecto y objeto en Hakhun es precisamente el elemento más complejo del puzzle.

En la estrategia incremental, el modelo con subconjuntos pequeños ($C(10,2)$, $C(10,3)$) produce traducciones degradadas por insuficiencia de evidencia para deducir el sistema pronominal de objeto. El rendimiento mejora en pasos intermedios pero decae en los pasos finales, posiblemente por sobrecarga del contexto del prompt. El humano en la condición incremental mostró el patrón opuesto: con solo dos frases ya identificó los elementos invariantes, y cada par adicional refinó hipótesis previas sin degradar las ya establecidas.

5. Desarrollo (Con ejemplos de código)

5.1. Inserción de código

A veces tendrás que insertar algún pedazo de código fuente para explicar algo relacionado con él. No sustituyas explicaciones con códigos enormes. Si pones algo de código en tu TFG que sea para demostrar algo o explicar alguna solución.

L^AT_EX te ayuda a escribir código de manera que su presentación tenga las marcas y tabulaciones propias de este tipo de texto. Para ello, debes poner el código que escribas DENTRO de un entorno que se llama “listings”. La plantilla ya tiene una serie de instrucciones para incluir el paquete “listings” y añadirle algunos modificadores por lo que no tienes que incluirlo tú. Simplemente, mete tu código en el entorno “lstlisting” y ya está. Puedes indicar el lenguaje en el que está escrito el código y así L^AT_EX lo mostrará mejor.

En el archivo *estiloscodigoprogramacion.tex* están definidos algunos lenguajes para mostrarlos con un diseño concreto, se pueden modificar para cambiar el coloreado del código, qué términos se ponen en negrita, etc. Si se quiere profundizar más en la función “listings” se puede consultar su manual en <http://osl.ugr.es/CTAN/macros/latex/contrib/listings/listings.pdf>, aunque hay mucha información en foros y blog’s que es más fácil de comprender.

Veamos un ejemplo en la figura 5.1:

```
\begin{lstlisting}[style=C, caption={ejemplo código C},label=C_code]
#include <stdio.h>
int main(int argc, char* argv[]) {
    puts("Hola mundo!");
}
\end{lstlisting}
```

El resultado será:

Código 5.1: ejemplo código C

```
1 #include <stdio.h>
2 // Comentario
3 int main(int argc, char* argv[]) {
4     puts("Hola mundo!");
5 }
```

Si lo quieres en color, está definido el estilo C-color en el archivo *estiloscodigoprogramacion.tex*, con algunos parámetros para mejorar la visualización:

```
\begin{lstlisting}[style=C-color, caption={ejemplo código C en color},label=C_code-color]
#include <stdio.h>
// Comentario
int main(int argc, char* argv[]) {
```

```
puts("Hola mundo!");
}
\end{lstlisting}
```

Código 5.2: ejemplo código C en color

```
1  #include <stdio.h>
2  // Comentario
3  int main(int argc, char* argv[]) {
4  puts("Hola mundo!");
5  }
```

Por supuesto, puedes mejorar esta presentación utilizando más modificadores. En la sección 5.2 se indican algunos detalles.

Otro ejemplo, ahora para mostrar código PHP, sería escribir en tu fichero $\text{\LaTeX}$ lo siguiente:

```
\begin{lstlisting}[style=PHP, caption={ejemplo código PHP},label=PHP_code]
/*
Ejemplo de código en PHP para escribir tu primer programa en este lenguaje
Copia este código en tu ordenador y ejecútalo
*/
<html>
  <head>
    <title>Prueba de PHP</title>
  </head>
  <body>
    <?php echo '<p>Hola Mundo</p>'; ?> //esto lo escribe TODO el mundo
  </body>
</html>
\end{lstlisting}
```

y el resultado es el siguiente:

Código 5.3: ejemplo código PHP

```
100 /*
101 Ejemplo de código en PHP para escribir tu primer programa en este lenguaje. Copia este código en tu ↵
    ↵ ordenador y ejecútalo
102 */
103 <html>
104 <head>
105   <title>Prueba de PHP</title>
106 </head>
107 <body>
108   <?php echo '<p>Hola Mundo</p>'; ?> //esto lo escribe TODO el mundo
109 </body>
110 </html>
```

O también en color:

Código 5.4: ejemplo código PHP

```
1 /*
2 Ejemplo de código en PHP para escribir tu primer programa en este lenguaje. Copia este código en tu ↵
    ↵ ordenador y ejecútalo
3 */
4 <html>
5 <head>
6   <title>Prueba de PHP</title>
```

```

7 </head>
8 <body>
9 <?php echo '<p>Hola Mundo</p>'; ?> //esto lo escribe TODO el mundo
10 </body>
11 </html>

```

Observa cómo L^AT_EX ha puesto los comentarios en gris y ajustado el código para que se muestre más claro.

A continuación se muestran otros ejemplos:

Código 5.5: ejemplo código Matlab en color

```

1 %% Code sections are highlighted.
2 % System command are supported...
3 !touch testFile.txt
4 A = [1, 2, 3;... %... as is line continuation.
5     4, 5, 6];
6 fid = fopen('testFile.text', 'w');
7 for k=1:10
8     fprintf(fid, '%6.2f \n', k)
9 end
10 x=1; %% this is just a comment, not the start of a section
11 % Context-sensitive keywords get highlighted correctly...
12 p = properties(person); %(here, properties is a function)
13 x = linspace(0,1,101);
14 y = x(end:-1:1);
15 % ... even in nonsensical code.
16 ]end()()(((end while { end )end )))end (end
17 %{
18     block comments are supported
19 %} even
20 runaway block comments are

```

Código 5.6: ejemplo código Matlab en blanco y negro

```

1 %% Code sections are highlighted.
2 % System command are supported...
3 !touch testFile.txt
4 A = [1, 2, 3;... %... as is line continuation.
5     4, 5, 6];
6 fid = fopen('testFile.text', 'w');
7 for k=1:10
8     fprintf(fid, '%6.2f \n', k)
9 end
10 x=1; %% this is just a comment, not the start of a section
11 % Context-sensitive keywords get highlighted correctly...
12 p = properties(person); %(here, properties is a function)
13 x = linspace(0,1,101);
14 y = x(end:-1:1);
15 % ... even in nonsensical code.
16 ]end()()(((end while { end )end )))end (end
17 %{
18     block comments are supported
19 %} even
20 runaway block comments are

```

Código 5.7: ejemplo código Python en color

```

1 class Example (object):
2     def __init__ (self, account, password):
3         """e.g. account = 'bob@example.com/test'
4             password = 'bigbob'
5         """
6
7         reg = telepathy.client.ManagerRegistry()
8         reg.LoadManagers()
9
10        # get the gabble Connection Manager
11        self.cm = cm = reg.GetManager('gabble')
12
13        # get the parameters required to make a Jabber connection
14        # begin ex.basics.dbus.language-bindings.python.methods.call
15        cm[CONNECTION_MANAGER].RequestConnection('jabber',
16        {
17            'account': account,
18            'password': password,
19        },
20        reply_handler = self.request_connection_cb,
21        error_handler = self.error_cb)
22        # end ex.basics.dbus.language-bindings.python.methods.call

```

Código 5.8: ejemplo código Python en blanco y negro

```

1 class Example (object):
2     def __init__ (self, account, password):
3         """e.g. account = 'bob@example.com/test'
4             password = 'bigbob'
5         """
6
7         reg = telepathy.client.ManagerRegistry()
8         reg.LoadManagers()
9
10        # get the gabble Connection Manager
11        self.cm = cm = reg.GetManager('gabble')
12
13        # get the parameters required to make a Jabber connection
14        # begin ex.basics.dbus.language-bindings.python.methods.call
15        cm[CONNECTION_MANAGER].RequestConnection('jabber',
16        {
17            'account': account,
18            'password': password,
19        },
20        reply_handler = self.request_connection_cb,
21        error_handler = self.error_cb)
22        # end ex.basics.dbus.language-bindings.python.methods.call

```

5.2. Usos y personalización

El texto que acompaña al código puedes incluirlo o no, también puedes decidir si el texto va numerado o no. A continuación se muestra como:

```

% Con esta línea el código no tendrá título
\begin{lstlisting}[style=Python]
micodigo
\end{lstlisting}

```

```
1 micodigo
```

```
% Con esta línea el código tendrá el título abajo
\begin{lstlisting}[style=Python, caption={Ejemplo de título abajo},captionpos=b]
micodigo
\end{lstlisting}
```

```
1 micodigo
```

Código 5.9: Ejemplo de título abajo

```
% Con esta línea el código tendrá título no numerado
\begin{lstlisting}[style=Python, title={Ejemplo de título no numerado}]
micodigo
\end{lstlisting}
```

Ejemplo de título no numerado

```
1 micodigo
```

```
% Con esta línea el código no tendrá las líneas numeradas
\begin{lstlisting}[style=Python,numbers=none, title={Ejemplo de código sin número de líneas}]
micodigo
sin
número
de
líneas
\end{lstlisting}
```

Ejemplo de código sin número de líneas

```
micodigo
sin
número
de
líneas
```

5.3. Importar archivos fuente

Existe la posibilidad de importar un archivo de código en lugar de copiar su contenido y pegarlo en $\text{\LaTeX}$.

Para realizarlo debes escribir:

```
\lstinputlisting[style=C++-color,caption={Archivo C++ importado}]{archivos/ejemplos/holamundo.cpp}
```

Y se importará con el formato establecido entre los '[']':

Código 5.10: Archivo C++ importado

```

1 #include <stdio.h>
2 int main()
3 {
4     // printf() displays the string inside quotation
5     printf("Hello, World!");
6     return 0;
7 }

```

A continuación se muestran otros ejemplos

\lstinputlisting[style=Python-color,caption={Archivo Py importado},label=importado_py]{archivos/↵
↵ ejemplos/holamundo.py}

Código 5.11: Archivo Py importado

```

1 #-----↵
2 # helloworld.py
3 #-----↵
4
5 import stdio
6
7 # Write 'Hello, World' to standard output.
8 stdio.writeln('Hello, World')
9
10 #-----↵
11
12 # python helloworld.py
13 # Hello, World

```

\lstinputlisting[style=Matlab-color,caption={Archivo Matlab importado},label=importado_m]{archivos/↵
↵ ejemplos/holamundo.m}

Código 5.12: Archivo Matlab importado

```

1 function y = hello_world %#codegen
2
3 y = 'Hello World!';
4
5 end
6 % Copyright 2010 The MathWorks, Inc.

```

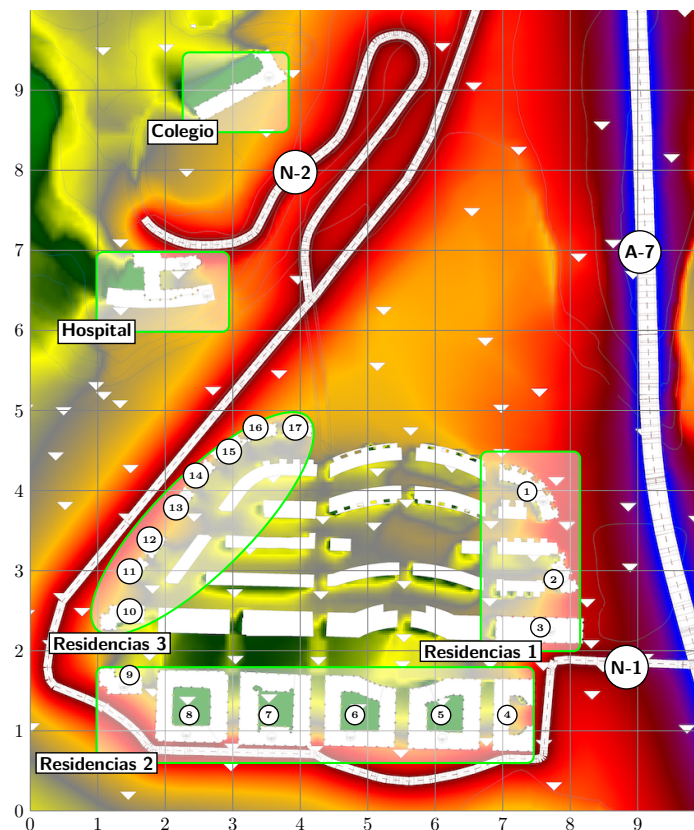

6. Resultados (Con ejemplos de gráficos)

6.1. Diagramas

Gracias al paquete *Tikz* se pueden incluir multitud de medios gráficos, diagramas, capas sobre imágenes, etc. Existen múltiples formas de realizarlo, para ello es recomendable consultar la guía de iniciación disponible aquí: <http://cremeronline.com/LaTeX/minimaltikz.pdf> y también el manual completo disponible aquí: <http://osl.ugr.es/CTAN/graphics/pgf/base/doc/pgfmanual.pdf>.

A continuación se muestran algunos ejemplos. Revisa el archivo .tex para ver cómo se utilizan.

Imagen a la que se le ha añadido cuadros y texto desde latex:



En muchas ocasiones es necesario realizar un diagrama de bloques, más abajo se muestra

un ejemplo de ello. En la red hay multitud de ejemplos que pueden ser fácilmente modificables para un fin concreto, como por ejemplo en esta web: <http://www.texample.net/tikz/examples/tag/block-diagrams/>.

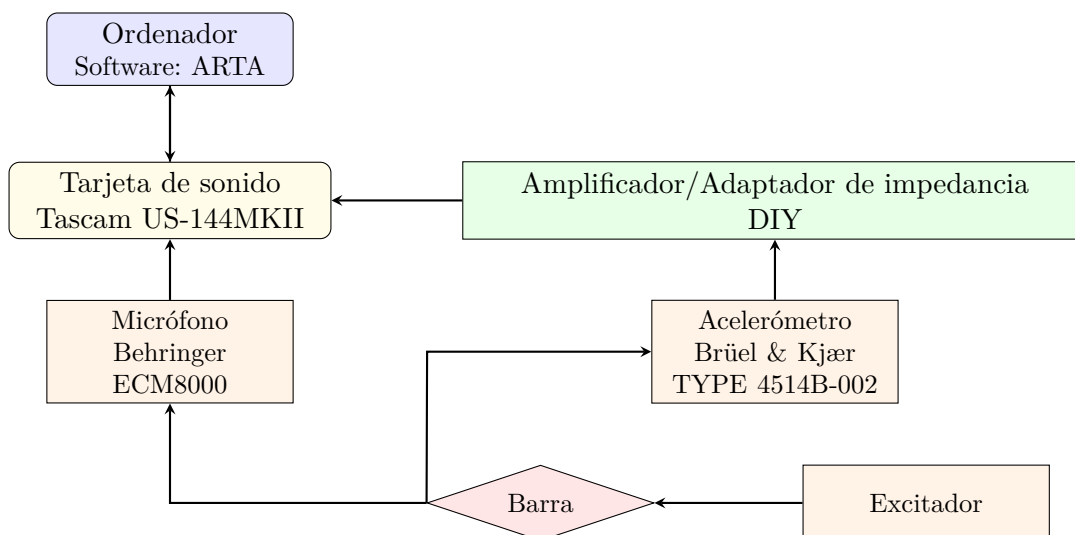


Figura 6.1: Diagrama realizado en latex con Tikz.

6.2. Gráficas

Existen múltiples formas de generar gráficas para latex. Hay disponibles herramientas como GeoGebra que dispone de la utilidad para exportar los gráficos en formato Tikz. También funciones para Matlab que genera las gráficas que muestra habitualmente pero en código para Tikz.

6.2.1. Línea

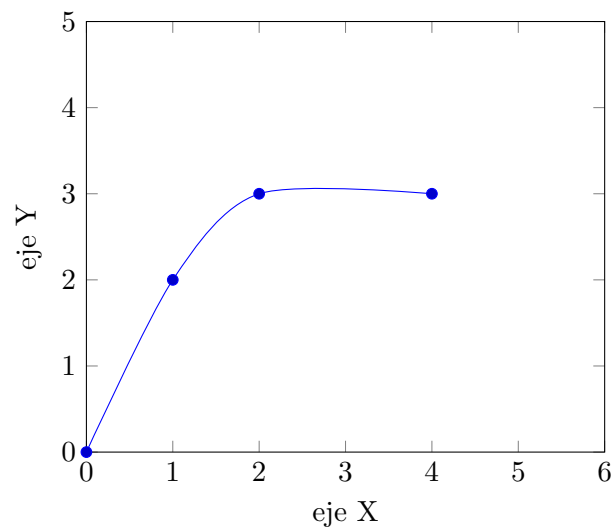
La forma más simple, aunque no sencilla cuando abarca muchos datos es la siguiente:

```

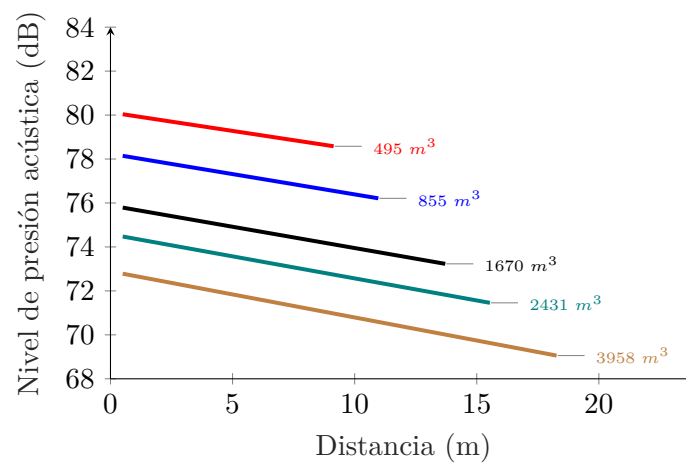
\begin{figure}[ht]
\centering
\begin{tikzpicture}
\begin{axis}
[ymin=0,ymax=5, % Límites del eje y
xmin=0,xmax=6, % Límites del eje x
ylabel= eje Y, % Nombre del eje y
xlabel= eje X] % Nombre del eje x
\addplot+[smooth] coordinates % Une los puntos curva suavizada
{(0,0) (1,2) (2,3 (4,3))}; % Puntos de la gráfica
\end{axis}
\end{tikzpicture}
\caption{Gráfica sencilla.}
\end{figure}

```

El resultado es el siguiente:

**Figura 6.2:** Gráfica sencilla.

Otro ejemplo, en este caso las líneas están calculadas directamente en LaTeX y después cada una tiene una anotación (el código se encuentra en el archivo `archivos/ejemplos/perjudicialesopticaentro.tex`).

**Figura 6.3:** OP/S003

6.2.2. Barras

Otro ejemplo es la gráfica de barras:

```
\begin{figure}[ht]
\centering
\begin{tikzpicture}
\begin{axis}[
ybar=12pt,
ymin=0,ymax=150,
xtick=data,
```

```

enlarge x limits={abs=2cm},
symbolic x coords={rubio, moreno},
bar width = 20pt,
ylabel= número,
xlabel= color de pelo,
ytick align=outside,
ytick pos=left,
major x tick style = transparent,
legend style={at={(0.04,0.96)},anchor=north west, font=\footnotesize, legend cell align=left,},
]
\addplot[ybar,fill=blue, area legend] coordinates {
(rubio,20)
(moreno,100)};
\addplot[ybar,fill=purple, area legend] coordinates {
(rubio,110)
(moreno,105)};
\legend{Chicos, Chicas}
\end{axis}
\end{tikzpicture}
\caption{Gráfica barras.}
\end{figure}

```

El resultado es el siguiente:

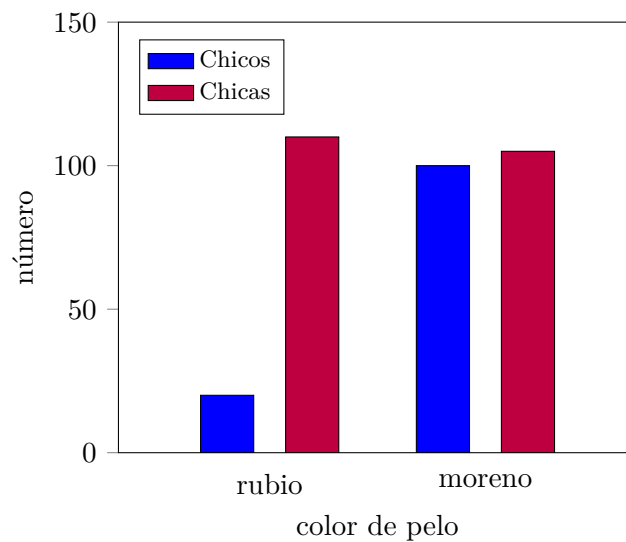


Figura 6.4: Gráfica barras.

6.2.3. Polar

Un ejemplo de gráfica polar semicircular (ver archivo `archivos/ejemplos/polarnorm.tex`):

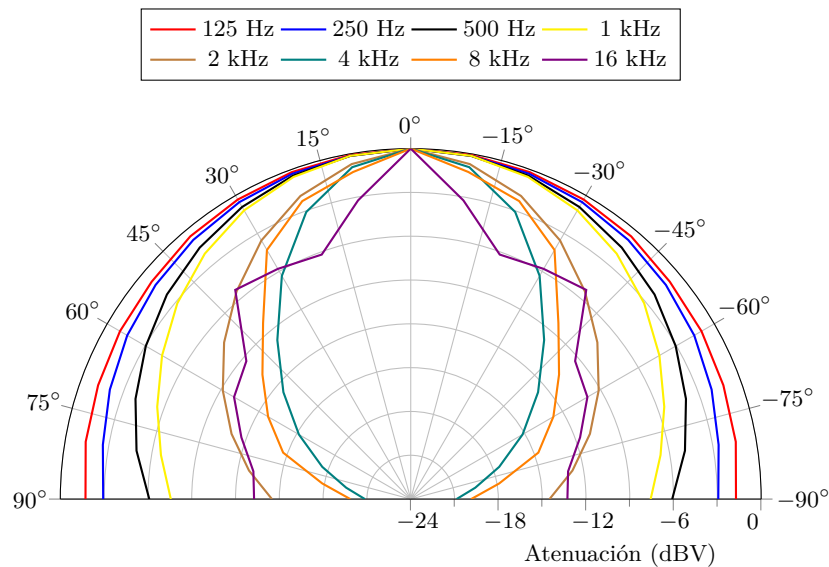


Figura 6.5: Directividad normalizada del altavoz (0 dBV en el eje).

6.3. Importados de MATLAB

Gracias a la herramienta *matlab2tikz* (<https://es.mathworks.com/matlabcentral/fileexchange/22022-matlab2tikz-matlab2tikz>) se pueden exportar las gráficas de cualquier tipo de Matlab a latex. Después de incluir los archivos de *matlab2tikz* se debe escribir una llamada después de crear la figura tal que:

Código 6.1: Ejemplo de llamada a *matlab2tikz*

```
1 fig = plot(x,y);
2 matlab2tikz('figurehandle',fig,'NombreArchivo.tex','height','5cm','width','13.5cm','strict',true,'↔
  ↳ showHiddenStrings',true,'showInfo',false)
```

Y para utilizar el archivo generado por la función en este documento:

```
\begin{figure}[ht]
\centering
{\scalefont{0.8}\input{archivos/ejemplos/ParedFina} }
\caption{Ejemplo de gráfica obtenida con matlab2tikz.}
\end{figure}
```

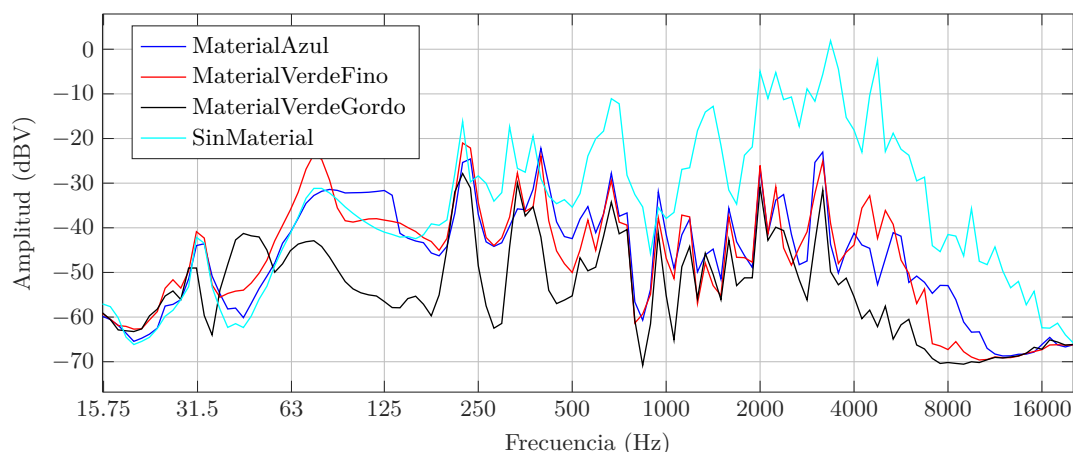


Figura 6.6: Ejemplo de gráfica obtenida con `matlab2tikz`.

Ejemplo de una gráfica 3D generada en Matlab y exportada por `matlab2tikz`:

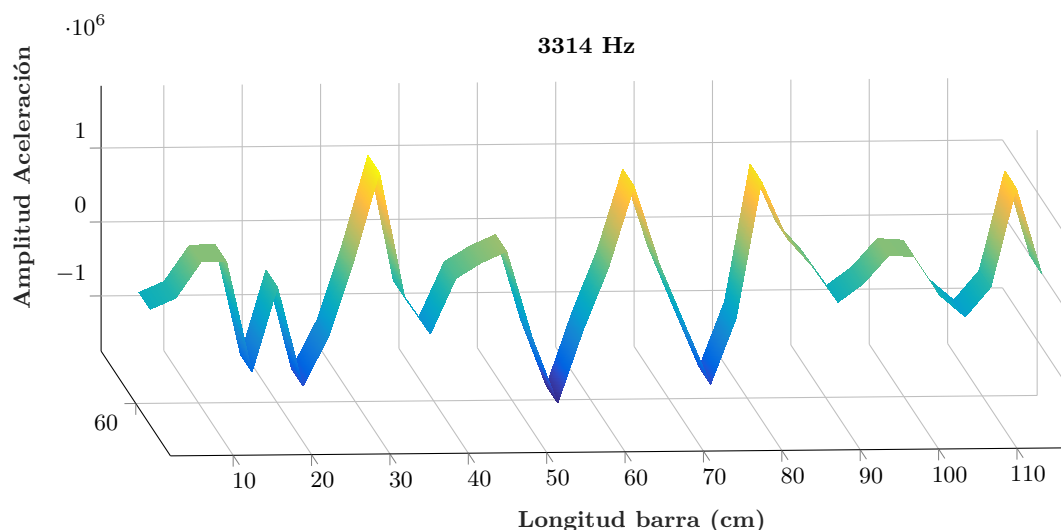


Figura 6.7: Amplitud de la aceleración en el modo número 8.

6.4. Ejemplo avanzado

El potencial del paquete *Tikz* es muy alto, se pueden realizar muchísimas cosas. En la red se facilitan muchos ejemplos para poder ver el funcionamiento y aprender. Existen hilos donde la gente publica sus mejores diseños de *Tikz* como en <https://tex.stackexchange.com/questions/158668/nice-scientific-pictures-show-off> o páginas donde facilitan muchas plantillas como <http://www.texample.net/tikz/examples/all/>.

Un ejemplo de lo que se puede llegar a conseguir es el siguiente:

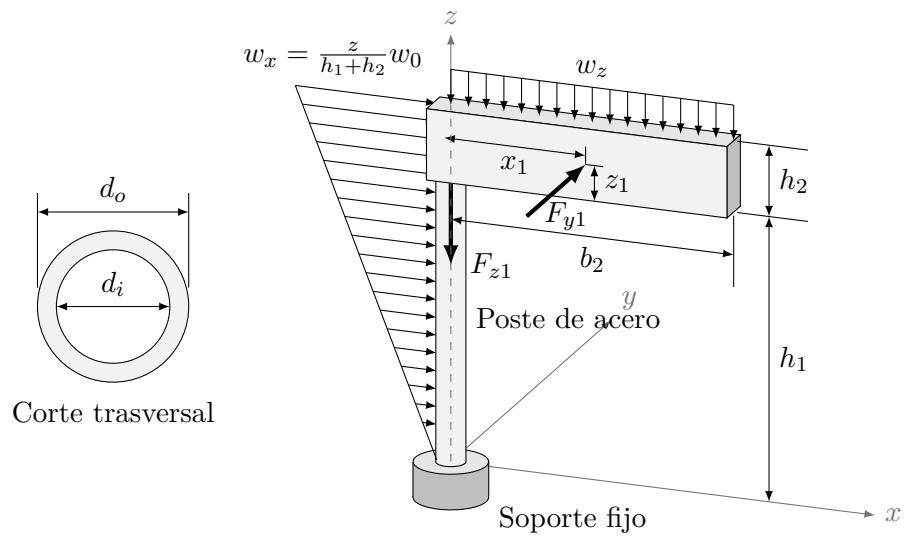


Figura 6.8: Señal realizada con Tikz, sin imágenes.

7. Conclusiones (Con ejemplos de matemáticas)

7.1. Matemáticas

En $\text{\LaTeX}$ se pueden mostrar ecuaciones de varias formas, cada una de ellas para un fin concreto.

Antes de ver algunas de estas formas hay que conocer cómo se escriben fórmulas matemáticas en $\text{\LaTeX}$. Una fuente de información completa es la siguiente: <https://en.wikibooks.org/wiki/LaTeX/Mathematics>. También existen herramientas online que permiten realizar ecuaciones mediante interfaz gráfica como <http://www.hostmath.com/>, <https://www.mathcha.io/editor> o <https://www.latex4technics.com/>

Para mostrar una ecuación numerada se debe utilizar:

```
\begin{equation}
\nabla\times{\mathbf H}=\left[\frac{1}{r}\frac{\partial}{\partial r}(rH_\theta)-\frac{1}{r}\frac{\partial H_r}{\partial\theta}\right]\hat{\mathbf z}
\label{ecuacion}
\end{equation}
```

$$\nabla \times \mathbf{H} = \left[\frac{1}{r} \frac{\partial}{\partial r} (r H_\theta) - \frac{1}{r} \frac{\partial H_r}{\partial \theta} \right] \hat{\mathbf{z}} \quad (7.1)$$

Si es necesario agrupar varias ecuaciones en un mismo índice se puede escribir del siguiente modo:

```
\begin{subequations}
\begin{eqnarray}
{\mathbf E}&=&E_z(r,\theta)\hat{\mathbf z}\label{ecu1} \\
{\mathbf H}&=&H_r(r,\theta)\hat{\mathbf r}+H_\theta(r,\theta)\hat{\mathbf \theta}\label{ecu2}
\end{eqnarray}
\end{subequations}
% Se incluye '&' entre la igualdad para centrar las ecuaciones desde el '='.
```

$$\mathbf{E} = E_z(r, \theta) \hat{\mathbf{z}} \quad (7.2a)$$

$$\mathbf{H} = H_r(r, \theta) \hat{\mathbf{r}} + H_\theta(r, \theta) \hat{\boldsymbol{\theta}} \quad (7.2b)$$

Otras dos formas que son las habituales en muchos lugares para incluir ecuaciones son:

Ejemplo de fórmula en línea con el texto `\int_{a}^{b} f(x)dx = F(b) - F(a)`, esta ecuación quedará dentro $\leftrightarrow$ del texto.

Esta otra, al utilizar dos `'\$'`, se generará en una línea nueva `\int_{a}^{b} f(x)dx = F(b) - F(a)`

Ejemplo de fórmula en línea con el texto $\int_a^b f(x)dx = F(b) - F(a)$, esta ecuación quedará dentro del texto.

Esta otra, al utilizar dos `'$'`, se generará en una línea nueva

$$\int_a^b f(x)dx = F(b) - F(a)$$

También se puede añadir información adicional a una ecuación con la función *condiciones* creada para esta plantilla:

```
\begin{equation}
\underset{z=z_0}{\mathrm{Res}}(f(z))=\frac{1}{(m-1)!}\lim_{z\rightarrow z_0}\left[\frac{d^{m-1}}{dz^{m-1}}[(z-z_0)^m f(z)]\right]
\end{equation}

\begin{condiciones}[donde:]
% Excepto 'Descripción y valor' el resto no es necesario el símbolo $para texto matemático.
% Item & Relación & Descripción o valor
m & \rightarrow & Es la multiplicidad del polo $z_0$ \\
z_0 & \rightarrow & Es la parte que se iguala a 0 con el polo. \\
f(z) & \rightarrow & Es la función contenida en la integral.
\end{condiciones}
```

$$\mathrm{Res}(f(z)) = \frac{1}{(m-1)!} \lim_{z \rightarrow z_0} \left[\frac{d^{m-1}}{dz^{m-1}} [(z - z_0)^m f(z)] \right] \quad (7.3)$$

donde: $m \rightarrow$ Es la multiplicidad del polo z_0

$z_0 \rightarrow$ Es la parte que se iguala a 0 con el polo.

$f(z) \rightarrow$ Es la función contenida en la integral.

Si lo que deseas es una ecuación alineada a la izquierda o derecha puedes hacerlo con lo siguiente (el `'&'` simple es utilizado para alinear las ecuaciones desde ese punto, los iguales):

```
% Alineado a la izquierda al incluir al final el doble '&&'
\begin{flalign}
y_{h_1} &= \begin{bmatrix} 6\cos(\sqrt{6}x) \\ -\sqrt{6}\sin(\sqrt{6}x) \end{bmatrix} e^x && \\
y_{h_2} &= \begin{bmatrix} 6\sin(\sqrt{6}x) \\ \sqrt{6}\cos(\sqrt{6}x) \end{bmatrix} e^x && \\
\end{flalign}

% Alineado a la derecha al incluir al inicio el doble '&&'
\begin{flalign}
&& y_{h_1} = \begin{bmatrix} 6\cos(\sqrt{6}x) \\ -\sqrt{6}\sin(\sqrt{6}x) \end{bmatrix} e^x \\
&& y_{h_2} = \begin{bmatrix} 6\sin(\sqrt{6}x) \\ \sqrt{6}\cos(\sqrt{6}x) \end{bmatrix} e^x \\
\end{flalign}
```

$$y_{h_1} = \begin{bmatrix} 6 \cos(\sqrt{6}x) \\ -\sqrt{6} \sin(\sqrt{6}x) \end{bmatrix} e^x \quad (7.4)$$

$$y_{h_2} = \begin{bmatrix} 6 \sin(\sqrt{6}x) \\ \sqrt{6} \cos(\sqrt{6}x) \end{bmatrix} e^x \quad (7.5)$$

$$y_{h_1} = \begin{bmatrix} 6 \cos(\sqrt{6}x) \\ -\sqrt{6} \sin(\sqrt{6}x) \end{bmatrix} e^x \quad (7.6)$$

$$y_{h_2} = \begin{bmatrix} 6 \sin(\sqrt{6}x) \\ \sqrt{6} \cos(\sqrt{6}x) \end{bmatrix} e^x \quad (7.7)$$

Tanto con la función utilizada en (7.1,7.3), como en (7.2a,7.2b) y en las anteriores, si se les incluye un '*' después de 'equation', 'subequation' o 'flalign', se elimina la numeración de las ecuaciones pero manteniendo el resto de características.

Bibliografía

- AI@Meta. (2024). *Llama 3 Model Card*. <https://github.com/meta-llama/llama3>.
- Bean, A. M., Hellsten, S., Mayne, H., Magomere, J., Chi, E. A., Chi, R., ... Kirk, H. R. (2024). LINGOLY: A Benchmark of Olympiad-Level Linguistic Reasoning Puzzles in Low-Resource and Extinct Languages. En *Advances in neural information processing systems (neurips 2024), track on datasets and benchmarks*. Descargado de <https://arxiv.org/abs/2406.06196>
- Bozhanov, B., y Derzhanski, I. (2013). Rosetta Stone Linguistic Problems. En *Proceedings of the fourth workshop on teaching nlp and cl* (pp. 1–8). Sofia, Bulgaria: Association for Computational Linguistics.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... others (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. Descargado de <https://arxiv.org/abs/2005.14165>
- Chi, N., Malchev, T., Kong, R., Chi, R., Huang, L., Chi, E., ... Radev, D. (2024). Modeling: A Novel Dataset for Testing Linguistic Reasoning in Language Models. En *Proceedings of the 6th workshop on research in computational linguistic typology and multilingual nlp* (pp. 113–119). St. Julian's, Malta: Association for Computational Linguistics.
- Choudhary, M., Srivatsa, K. A., Aeron, G., Bhattacharya, A. R., Dinh, D. K. D., Hanif, I. A., ... Choudhary, M. (2025). unweiLing: What Makes Linguistics Olympiad Puzzles Tricky for LLMs? En *Proceedings of the conference on language modeling (colm 2025)*. Descargado de <https://arxiv.org/abs/2508.11260> (Analiza 629 problemas en 41 lenguas de baja densidad. Concluye que la complejidad morfológica es el factor más determinante del fracaso de los LLMs)
- Derzhanski, I., y Payne, T. E. (2010). The Linguistics Olympiads: Academic Competitions in Linguistics for Secondary School Students. En *Linguistics at school: Language awareness in primary and secondary education* (pp. 213–226). Cambridge University Press.
- Eberhard, D. M., Simons, G. F., y Fennig, C. D. (2020). *Ethnologue: Languages of the World*. Dallas, Texas: SIL International. Descargado de <https://www.ethnologue.com> (23rd edition)
- Joshi, P., Santy, S., Budhiraja, A., Bali, K., y Choudhury, M. (2020). The State and Fate of Linguistic Diversity and Inclusion in the NLP World. En *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 6282–6293). doi: 10.18653/v1/2020.acl-main.560

- Majmudar, N., y Filatova, E. (2025). Can LLMs Solve and Generate Linguistic Olympiad Puzzles? *arXiv preprint arXiv:2509.21820*. Descargado de <https://arxiv.org/abs/2509.21820> (Trabajo más reciente del campo. Demuestra que o1 supera a humanos en la mayoría de tipos de puzzle excepto en sistemas de escritura y lenguas poco estudiadas)
- Papineni, K., Roukos, S., Ward, T., y Zhu, W.-J. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. En *Proceedings of the 40th annual meeting of the association for computational linguistics* (pp. 311–318). doi: 10.3115/1073083.1073135
- Popović, M. (2015). chrF: Character n-gram F-score for Automatic MT Evaluation. En *Proceedings of the tenth workshop on statistical machine translation* (pp. 392–395). Lisbon, Portugal: Association for Computational Linguistics. doi: 10.18653/v1/W15-3049
- Popović, M. (2017). chrF++: Words Helping Character n-grams. En *Proceedings of the second conference on machine translation* (pp. 612–618). Copenhagen, Denmark: Association for Computational Linguistics. doi: 10.18653/v1/W17-4726
- Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. En *Proceedings of the third conference on machine translation: Research papers* (pp. 186–191). Association for Computational Linguistics. doi: 10.18653/v1/W18-6319
- Ramji, R., y Ramji, K. (2024). Inductive Linguistic Reasoning with Large Language Models. *arXiv preprint arXiv:2412.17819*. Descargado de <https://arxiv.org/abs/2412.17819> (Propone prompting analógico: el modelo genera ejemplos auxiliares de lenguas relacionadas para guiar el razonamiento inductivo)
- Sánchez, E., Alastruey, B., Ropers, C., Stenetorp, P., Artetxe, M., y Costa-jussà, M. R. (2024). Linguini: A Benchmark for Language-Agnostic Linguistic Reasoning. En *arxiv preprint arxiv:2409.12126*. Descargado de <https://arxiv.org/abs/2409.12126> (Meta AI / University College London / Universidad del País Vasco)
- Tanzer, G., Suzgun, M., Visser, E., Jurafsky, D., y Melas-Kyriazi, L. (2024). A Benchmark for Learning to Translate a New Language from One Grammar Book. En *Proceedings of the 12th international conference on learning representations (iclr 2024)*. Descargado de <https://arxiv.org/abs/2309.16575>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., ... Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35. Descargado de <https://arxiv.org/abs/2201.11903>
- Zhu, H., Liang, Y., Xu, W., y Xu, H. (2025). Evaluating Large Language Models for In-Context Learning of Linguistic Patterns in Unseen Low Resource Languages. En *Proceedings of the first workshop on language models for low-resource languages (lo-reslm) at acl 2025*. Association for Computational Linguistics. Descargado de <https://aclanthology.org/2025.loreslm-1.31>
- Şahin, G. G., Kementchedjhieva, Y., Rust, P., y Gurevych, I. (2020). PuzzLing Machines: A Challenge on Learning From Small Data. En *Proceedings of the 58th annual meeting*
-

of the association for computational linguistics (pp. 1241–1254). Online: Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.115

A. Anexo I

Aquí vendría el anexo I

B. Páginas horizontales

Aquí se muestra cómo incluir páginas en horizontal.
Esta página está en vertical

Esta página está en horizontal

Esta página también está en horizontal

Esta página está de nuevo en vertical

C. Importar PDF

A continuación se muestra una página importada de un PDF externo. Observar los comentarios en el código de este anexo para más información. También puedes leer el manual con todas las opciones en <http://osl.ugr.es/CTAN/macros/latex/contrib/pdfpages/pdfpages.pdf>.

Alicante

15 DE MARZO DE 2007

Expediente número

Referencia del petionario

AYUNTAMIENTO DE ALICANTE

Departamento de Medio Ambiente

C/San Nicolás, nº 2, 4º

03001 ALICANTE

Contacto: Juan Luís Beresaluze

DOCUMENTO DE SÍNTESIS

***ELABORACIÓN DEL MAPA ACÚSTICO MUNICIPAL DE LA CIUDAD DE
ALICANTE***

Fecha de realización del estudio: MAYO 2005 – MARZO 2007